

Microscopic Traffic Simulation

Special topics on model calibration

Alessandro Scalese

March 13, 2026

Contents

1. Introduction	3
2. Calibration	5
2.1. Scenario analysis	5
3. Dynamic applications	6
Bibliography	8

1. Introduction

Traffic simulation models are invaluable tools in traffic planning and management. Traffic forecasts are one of the foremost information tools used by decision makers in transportation matters such as transportation policies and infrastructure investments, with significant impacts on communities and society as a whole.

In the early 2000s, Flyvbjerg et al (INSERT REFS) extensively analyzed the economic and traffic impacts of infrastructure projects undertaken in the previous decades. Among their findings, they highlighted that more than 50% of all projects reported differences in the forecasted and actual traffic demand above 20%, and that over 90% of the projects incurred in some measure of cost escalation, meaning the final cost was higher than the initial estimates. To top this off, they also underline that these — do not demonstrate — trends in their analysis period (ranging from the 1930s to the end of the 20th century).

This alone already does not paint a — landscape: inaccurate models can lead to possibly wrong or at least not optimal choices in transportation policies and infrastructure design, whose cost is more often than not underestimated, leading to — of public money.

One of the reasons behind the inaccuracy of traffic forecasts is the complexity of traffic modeling itself: a complete traffic model must account for several — components, from the microscopic variables that influence driving behavior, to the drivers of route choice and pathing, all the way up to the definition of the overarching traffic flows. Each of these sub-components can be modelled in different ways, with different complexities and parameters, and each layer introduces errors and indirections which contribute to the inaccuracy of the final model. Furthermore, the lack of accurate data presents another challenge: the OD trips, which define the flows across the network zones, are not known a priori, but either have to be modeled from socio-economic and topological variables (as done in the four-step model for traffic assignment INSERT REFS) or estimated from traffic measurements (data-driven approach). These measurements are taken from traffic sensors such as induction loops or cameras, and their coverage of city networks is often sparse and uneven, which adds on the uncertainty of the measurements themselves.

Modeling traffic is a complex endeavor: in the last hundred years, countless models have been developed to try and model human choice and driving behavior in a satisfying manner. Some of the models include GHR, IDM, — , all car following models etc. Even if we look at traffic on a bigger scale (meso- and macro-scopic simulations) a lot of the original problems remain: how can we model the way humans take pathing decisions? What are the variables and parameters that come at play?

the most complex (and accurate) models in use today require accurate calibration of countless variables just to describe human behavior, which is not a universal constant but changes based on the culture, topology, — of the problem we consider.

In addition to this, we have the constant and sometimes insurmountable lack of data: even knowing the whereabouts (origin and destination) of trips in a specific city is a challenge: in the most common example of traffic assignment, which is the four step model, this data that should be the most basic input for our problem has to be modeled from somewhere. This

introduces layers upon layers of indirections and assumptions that we have to make, each of which contributes non-insignificantly to the inaccuracy of the final model.

REF find something about the data quality

This is exacerbated by the lack of quality data in sufficient quantities. In most cities, when sensors are present (which is often not the case for small and medium cities), the network coverage is often sparse and uneven, leading to more difficulties.

What we call calibration here is the optimization of the model parameters to obtain from our simulation results that resemble the real traffic scenarios as closely as possible. In our case, these parameters reduce to the OD pairs that describe the number of trips between each origin and demand zone. As the number of OD pairs increases quadratically with the number of zones in our problem, it is easy to understand that even for smallish networks, we have a huge number of parameters to calibrate.

2. Calibration

2.1. Scenario analysis

We are tasked with calibrating the origin-demand matrix for the morning peak-hour period (8-9am) for a reduced network from the city of Aachen, in Germany. The network has been divided in twenty-three TAZes (Traffic Assignment Zones), which define the origin and destination of all trips on the network. This subdivision results in a total of 529 calibrable parameters (OD pairs).

The network also presents five hundred ninety nine loop detectors, which will be used in the simulation to record the counts, speeds and densities on various links on the network. While this might seem like a high number of sensors, we have to consider that all these sensors only cover around 15% of the network (which consists of more than four thousand individual links, or edges).

We also have a collection of the results of one hundred simulations, which we will need to use to determine the number of simulation replications required to account for the simulator intrinsic stochasticity. Indeed, our simulator of choice (SUMO with mesoscopic simulations enabled) is inherently stochastic, meaning that some of the variables used to model individual driving behavior are treated as random variables and therefore cause the results of multiple simulations on the same input data to give potentially different results. This data helps us quantify these differences and understand what sensors give us relevant (i.e. statistically significant results) and for how many simulations (this means we have to run the simulation x number of times and then average the results to get meaningful results).

The next task requires us to explore the viability and applicability of multiple goodness of fit functions to our problem, and to analyze and explore the input space to find a more solution (historic, a priori parameters). This initial solution will then be used as input for our calibration / optimization process. The optimization process uses the SPSA (Simultaneous Perturbation Stochastic Approximation) by Spall et al REF to calibrate the input OD matrix. The algorithm's hyperparameters are optimized themselves using an automatic search in the parameter space, powered by the Optuna package in python, which offers a plug-in system to explore it via a Parzen Tree (bayesian optimization etc). We added a slight change to the basic spsa approach: we used common random number generation to run simulation for both the plus and minus perturbations with the same seeds, which should in theory at least help isolate the effects of the perturbations from the simulator's stochasticity. This proved (empirically) to speed up the algorithm convergence.

After this initial optimization approach, we also developed following REF a PC-SPSA algorithm, which leverages Principal Component Analysis of the historical data to reduce the problem space (in our case, from 530 parameters to around 30), which speeds up optimization and also yields better results. The tuning of the hyperparameters was, again, done using optuna in an automatic search approach.

3. Dynamic applications

The implementation of dynamic dwell times more closely reflects reality compared to fixed stop times. In reality, dwell times can vary significantly between stops, due to varying number of people boarding / alighting the bus, and also due to some random human behavior (think for example about people running for the bus which then waits for them, or people almost missing their stop that request the driver to open the door for them).

However, dynamic dwell times also mean that the total travel time of the bus can vary significantly, making timetables inaccurate: this is one of the many reasons behind the — of digital timetables at modern bus stops, which can show a more accurate estimate of the bus ETA compared to fixed timetables which can only take into consideration an average of the historical data. On the environmental side, at least in our simulation, dynamic dwell times lead to a general reduction of the idle time of the bus, therefore causing a reduction in fuel consumption (and so of the emission of pollutants and CO₂). So: travel time generally down, reliability up

Request stops aim to reduce travel times (and idle times) even further as useless stops are skipped directly (while in the ddt scenarios we still stopped for around 15 seconds at each stop, even if nobody needed to board or alight). Should I talk about the technical difficulties of the implementation? E.g. understanding when to compute the number of people alighting, boarding etc

In general, time travel reduction due to sometimes skipping one of the stops -> the one with fewer requests. This should also lead to a reduction of fuel consumption and emissions compared to ddt -> understand why it is not shown in the graphs

What I need to do is run the simulation maybe like 4-5 times and then average the results. Violin plots should be perfect for this

Public transport prioritization: application to a single junction in the whole network -> if this has good results, think about what would happen with multiple implementations across the network.

Analyze the base program to understand the available phases and how we can play around with them:

- phase bus
- phase “major” -> should probably call sidestream or something
- phase “major-left” -> people that need to turn
- interstage

We place a detector far enough (considering that there is a bus stop before the TL -> bad design) and we try to account for its stop time (V2I hypothesis). After the bus decides if it is going to stop or not, then we apply the logic:

- if phase bus
 - if the bus makes it before the end of the max green, do nothing
 - otherwise,
 - if we have enough time to run a quick cycle (min green times etc) -> do that
 - otherwise, extend green time

- otherwise,
 - if phase is not major
 - wait
 - otherwise,
 - try to extend the major phase as long as possible to avoid disrupting the flow and only change when the bus is approaching allowing for a bit of time to let the queue dissipate

Instead of a simple “green time compensation” mechanism, we are going to implement a system that brings the tl back in sync with the adjacent one -> as we found empirically that prioritization would cause spillover and queues there due to people unable to occupy the junction in time. This is done by extending the major phase time and reducing the duration of the bus phase to bring them back in sync.

Very good results: significant reduction in travel times already with just one prioritized traffic light. Good reason to try and implement this in other junctions and for other bus lines as well, good pilot project.

Bibliography